



Proyecto Semana 05: API con SQLAlchemy ORM



Tu Dominio Asignado

Dominio: [El instructor te asignará tu dominio único]



IMPORTANTE: Cada aprendiz trabaja sobre un dominio diferente.



Ejemplo Genérico de Referencia

Los ejemplos usan **"Warehouse"** (Almacén) que **NO** está en el pool. **Debes adaptar TODO a tu dominio asignado.**

Concepto	Ejemplo Genérico	Adapta a tu Dominio
Primary Entity	Item	{YourEntity}
Secondary Entity	Supplier	{YourRelated}
Relationship	N:1 (many items per supplier)	{your_relationship}



Objetivo

Construir una **API REST completa** usando SQLAlchemy ORM con SQLite, implementando relaciones 1:N entre dos entidades de tu dominio.



Requisitos Funcionales (Adapta a tu Dominio)

Entities (Mínimo 2 relacionadas)

Secondary Entity (el "uno" de la relación):

```
# Ejemplo genérico (Warehouse - Supplier)
class Supplier(Base):
    __tablename__ = "suppliers"

    id: Mapped[int] = mapped_column(primary_key=True)
    code: Mapped[str] = mapped_column(String(20), unique=True)
    name: Mapped[str] = mapped_column(String(100))
    email: Mapped[str] = mapped_column(String(100), unique=True)
    contact_name: Mapped[str | None]
    is_active: Mapped[bool] = mapped_column(default=True)
    created_at: Mapped[datetime]

    # Relationship
    items: Mapped[list["Item"]] = relationship(back_populates="supplier")
```

Primary Entity (el "muchos" de la relación):

```
# Ejemplo genérico (Warehouse - Item)
class Item(Base):
    __tablename__ = "items"

    id: Mapped[int] = mapped_column(primary_key=True)
    sku: Mapped[str] = mapped_column(String(20), unique=True)
    name: Mapped[str] = mapped_column(String(200))
```

```
supplier_id: Mapped[int] = mapped_column(ForeignKey("suppliers.id"))
quantity: Mapped[int]
status: Mapped[ItemStatus]
created_at: Mapped[datetime]

# Relationship
supplier: Mapped["Supplier"] = relationship(back_populates="items")
```

Endpoints

Secondary Entity (Supplier):

Método	Endpoint	Descripción
POST	/suppliers/	Crear
GET	/suppliers/	Listar con paginación
GET	/suppliers/{id}	Obtener con sus items
PATCH	/suppliers/{id}	Actualizar
DELETE	/suppliers/{id}	Eliminar (si no tiene items)

Primary Entity (Item):

Método	Endpoint	Descripción
POST	/items/	Crear (validar FK)
GET	/items/	Listar (con filtro por supplier)
GET	/items/{id}	Obtener con supplier
PATCH	/items/{id}	Actualizar
DELETE	/items/{id}	Eliminar

 Estructura del Proyecto

```
starter/
├─ main.py
├─ database.py
├─ models/
│   ├── __init__.py
│   ├── supplier.py
│   └─ item.py
├─ schemas/
│   ├── __init__.py
│   ├── supplier.py
│   └─ item.py
├─ routers/
│   ├── __init__.py
│   ├── suppliers.py
│   └─ items.py
├─ alembic/
└─ pyproject.toml
```

└─ Dockerfile
└─ docker-compose.yml

✓ Criterios de Evaluación

Criterio	Puntos
Funcionalidad (40%)	
CRUD ambas entidades	15
Relación 1:N funciona	15
Alembic migraciones	10
Adaptación al Dominio (35%)	
Entidades coherentes con negocio	12
Relación lógica	13
Originalidad (no copia ejemplo)	10
Calidad del Código (25%)	
Models SQLAlchemy correctos	10
Eager/Lazy loading apropiado	10
Código limpio	5
Total	100

⚠ Política Anticopia

- ✗ **No copies** el ejemplo genérico "Supplier/Item"
- ✓ **Diseña** entidades específicas de tu dominio
- ✓ **Crea** relaciones lógicas para tu negocio

📖 Recursos

- [SQLAlchemy 2.0](#)
- [FastAPI + SQLAlchemy](#)
- [Pool de Dominios](#)

Tiempo estimado: 3 horas

[← Volver a Prácticas](#) | [Recursos →](#)